1. Цель работы

Реализовать прототип ML-пайплайна для сервиса, разделяющего песню (аудиофайл) на отдельные дорожки (стемы), и оценить его качество.

2. Задачи

1) Выбрать открытую модель для разделения аудио и реализовать её инференс для получения 4 стемов: vocals (вокалы), drums (ударные), bass (бас), other (остальное);
2) Добавить простой REST-эндпоинт, принимающий аудиофайл и возвращающий ссылки на 4 стема. Реализовать обработку исключений и асинхронную обработку;
3) Оптимизировать инференс: разделение аудиофайла на чанки, использование GPU;
4) Добавить ограничение времени обработки и визуализацию спектрограмм;
5) Обработать треки из подвыборки публичного датасета MUSDB18, проанализировать качество разделения (метрики SIR, SDR, SAR; время разделения), предложить возможные улучшения и сделать выводы.

3. Стэк и оборудование

Программный код написан на языке программирования Python3 в средах разработки VS Code и Jupyter Notebook. В качестве исследуемой предобученной модели была выбрана demucs, работа с которой осуществлялась при помощи пакетов demucs и PyTorch. Работа с аудио выполнена с использованием библиотек torchaudio, librosa и museval. WEB-часть реализована на фреймворке FastAPI. Анализ данных выполнен на классическом стеке NumPy, Pandas, matplotlib. Отчет написан средствами пакета программ Microsoft Office.

Работа выполнялась на персональном ноутбуке со следующими характеристиками.

Процессор AMD Ryzen 5 3500U with Radeon Vega Mobile Gfx 2.10 GHz

Оперативная память 6,00 ГБ

Память 477 GB SSD INTEL SSDPEKNW512G8

Видеоадаптер NVIDIA GeForce GTX 1050 (4 GB), AMD Radeon(TM) Vega 8 Graphics (1010 MB)

4. Ход работы

4.1. Разработка

Модель Demucs была выбрана так как она открыта, написана на Torch (следовательно, отлично совместима и нет проблем с GPU) и известна. Примеры запусков

можно найти в README. Были выполнены все задачи, поставленные по этой части работы, дополнительно реализована асинхронная обработка и обработка ошибок.

Обрезка тишины на границах не была реализована, так как это скорее принесет вред при реальной задаче, возникающей в работе с DAW: собьется таймлайн и восстановить его будет крайне сложно. Кэширование результата, Docker-контейнеризация и Фронтенд-часть не были реализованы за неимением времени. Так как это лишь прототип, не было выполнено и разделение проекта на многофайловую структуру.

4.2. Анализ

В качестве валидационной подвыборки были выбраны 6 треков из папки test датасета MUSDB18 (из теории известно, что 6 точек достаточно для адекватного построения линейной регрессии: ясно, что для зависимости, ожидающейся константной, это так же выполнено) и 1 из папки train (для оценки качества модели на обучающем датасете). Результаты приведены в таблице 1.

SIR (Signal-Interference Ratio, отношение сигнала к “проникновениям”, утечкам, bleeding) показывает насколько хорошо происходит непосредственно разделение песни на стемы (чем значение выше, тем лучше; отрицательное число говорит о том, что ошибка громче самого сигнала). SAR (Signal-Artefacts Ratio, сигнал-артефакты) показывает, насколько много алгоритм вносит слышимых артефактов (призвуков) в аудио, то есть насколько сигнал громче шумов. SDR (Signal-Distortion Ratio, сигнал-искажения) показывает громкость искажений, привносимых алгоритмом в сигнал, слышимых, например, как перегруз.

Таблица 1. *Метрики demucs.*

Номер трека	**Стэм**	**SIR**, dB	SAR, dB	SDR, dB
1	vocals	13,56	13,43	8,92
	drums	12,76	8,29	6,67
	bass	10,53	19,6	11,88
	other	10,08	5,47	4,72
2	vocals	18,05	12,81	9,46
	drums	9,36	16,72	7,00
	bass	4,50	19,10	10,51
	other	16,54	7,13	7,95
3	vocals	12,76	-2,75	4,83
	drums	19,41	2,08	12,26
	bass	-14,68	28,98	9,67
	other	12,04	-12,30	6,54
4	vocals	9,73	15,83	4,58
	drums	17,30	20,77	9,95
	bass	21,21	26,14	12,94
	other	15,23	9,92	5,65
5	vocals	19,05	19,86	9,70
	drums	12,21	19,06	7,27

Номер трека	Стэм	**SIR**, dB	**SAR**, dB	**SDR**, dB
5	bass	16,81	14,67	11,17
	other	11,18	11,36	5,64
6	vocals	18,68	-0,77	10,04
	drums	25,53	-12,08	-2,63
	bass	-10,47	10,73	11,69
	other	1,19	-18,76	0,89
train	vocals	11,80	16,79	5,47
	drums	4,33	15,82	2,82
	bass	25,60	21,97	17,80
	other	19,85	15,18	9,97

Позволим себе считать, что все ошибки в разделении песни не зависят от самого ее содержания и являются равновероятными в этом смысле. Это, конечно, весьма грубо, так как очевидно, что больше сложных инструментов разделить труднее, чем мало простых, например. Отметим лишь, что за счет разбиения входа модели на чанки удалось с хорошей точностью избежать зависимости от времени трека (однако чем больше чанков, тем больше границ между ними, где могут быть естественные ошибки на краях).

В таком приближении можем усреднить каждую метрику, относящуюся к каждому стему, по всем песням. Результаты приведены в таблице 2. Учтем так же среднеквадратичное отклонение величин.

Таблица 2. *Усредненные метрики.*

Стэм	**SIR**, dB	**SAR**, dB	**SDR**, dB
vocals	14,80 ± 3,74	10,74 ± 8,87	7,57 ± 2,48
drums	14,41 ± 6,97	10,09 ± 11,76	6,19 ± 4,87
bass	7,64 ± 15,46	20,17 ± 6,29	12,24 ± 2,66
other	12,30 ± 5,96	2,57 ± 12,88	5,91 ± 2,82

Метрики на валидации имеют в среднем хорошие значения, которые сильно больше нуля, однако в реальных данных наблюдаются некоторые выбросы, связанные, предположительно, с содержанием песен и ошибках на границах чанков. Как и ожидалось, стем other в среднем оказывается сложнее для модели, так как там много разных, часто редких, инструментов, в отличие от достаточно неизменного вокала. Модель не очень устойчива к работе с басом и может вносить артефакты в дорожку барабанов. Видно так же, что модель в среднем пока склонна работать так, чтобы занижать какую-то метрику в силу другой.

Метрики на обучающей подвыборке очень близки к валидационным, говоря о хорошем обучении модели - нет переобучения, нет недообучения. Видно, однако, что имеет смысл добавить больше стемов – струнные, синты, перкуссия, эффекты, пространство. Это определенно поможет разгрузить стем other и улучшить модель.

Среднее время инференса занимает примерно 4 минуты 17 секунд, что связано со средним железом, выбранном для данной работы, и тем, что на GPU так же активно параллельно обучалась другая модель.

Пример спектрограмм приведен на рисунке 1.

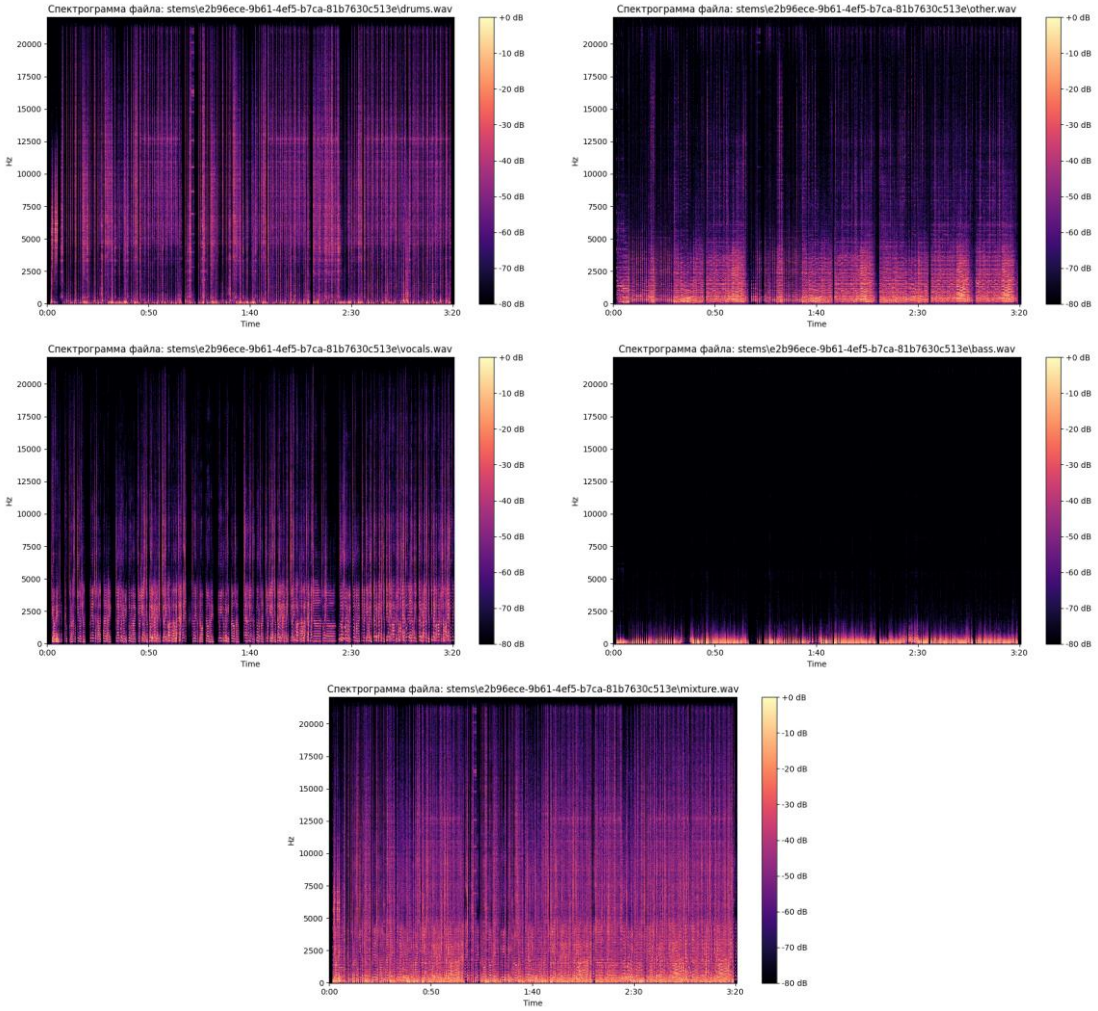

Рис. 1. *Спектрограммы трека и его стемов.*

4.3. Предложения

1) Как было отмечено, следует добавить больше стемов. Особое внимание нужно обратить на работу модели с пространством (реверберация, дилэи), которые часто даже выносятся в отдельные группы при работе с DAW. Возможно имеет смысл даже давать модели самой решить, на какие дорожки разбивать определенную песню, решая задачу кластеризации;
2) Хотелось бы, чтобы модель была инвариантна к тональности входного аудио: если подать на вход две песни, отличающиеся лишь высотой (pitch), то модель справлялась одинаково хорошо в обоих случаях. Это можно легко реализовать аугментацией данных. Скорее всего это так же увеличило бы общую точность модели.

5. Выводы

1) Были выполнены все поставленные задачи, реализованы ключевые элементы проекта ML-пайплайна, объяснено почему что-то не было реализовано;
2) Качество разделения песни на стемы моделью demucs было оценено на относительно репрезентативной подвыборке датасета MUSDB18, результаты оказались хорошими;
3) Предложены варианты улучшения архитектуры demucs: добавление задачи кластеризации, аугментация данных.